



Escuela
Politécnica
Superior

Aplicación colaborativa para propiciar el cuidado del medio ambiente



Grado en Ingeniería Informática

Trabajo Fin de Grado

Autor:

Erardo Aldana Pessoa

Tutor:

Dr. José Luis Sánchez Romero

Junio 2026



Universitat d'Alacant
Universidad de Alicante

Nota sobre este borrador

Este documento es un **borrador parcial** de la memoria del Trabajo Fin de Grado, preparado para una primera revisión por parte del tutor antes de la entrega definitiva (22 de mayo de 2026). Su objetivo es validar la estructura, el enfoque y el alcance del proyecto.

Capítulos incluidos en este borrador

- **Resumen y abstract** en español e inglés.
- **Capítulo 1 — Introducción:** motivación, objetivos formales y declaración explícita del uso de IA.
- **Capítulo 2 — Estado del arte:** análisis de cinco aplicaciones de referencia, comparativa funcional y comparativa técnica de las tecnologías evaluadas.
- **Capítulo 3 — Análisis y diseño:** requisitos funcionales (46 RF) y no funcionales (12 RNF), actores, casos de uso, arquitectura, modelo de datos (13 entidades), flujo de estados y diseño de la API REST.
- **Capítulo 6 — Conclusiones:** logros, limitaciones honestas y trabajo futuro.

Capítulos en redacción (no incluidos aquí)

- **Capítulo 4 — Implementación:** desarrollo por sprints, decisiones técnicas, extractos de código. Listo en la versión completa, omitido aquí para no saturar la primera revisión.
- **Capítulo 5 — Evaluación:** resultados de pruebas, validación de requisitos. Pendiente de incorporar las pruebas de usabilidad con tres usuarios reales.
- **Anexos:** guía de instalación, manual de usuario y modelo de negocio.

Pendiente de incorporar antes de la entrega final

- Capturas de pantalla reales de la aplicación (8 capturas previstas).
- Fichas detalladas de las cinco apps del estado del arte tras pruebas personales.
- Resultados cuantitativos de las pruebas de usabilidad.
- Texto final de los agradecimientos.

Cuestiones para la próxima tutoría

- Confirmación del estilo de citas (la plantilla usa APA 7).
 - Confirmación de la modalidad de defensa.
 - Decisión sobre publicación en RUA.
 - Cualquier observación sobre alcance, estructura o correcciones a aplicar antes de la entrega.
-

Erardo Aldana Pessoa
Convocatoria C3 — Curso 2025–2026

Resumen

EcoAlerta es una aplicación web progresiva (PWA) concebida como red social cívica especializada en medio ambiente. Permite a cualquier ciudadano reportar incidencias medioambientales (vertidos ilegales, contaminación, incendios forestales, animales abandonados o maltratados, daños a espacios naturales, entre otras) con foto y GPS, las clasifica por categoría y severidad, y las delega a la entidad responsable correcta (ayuntamiento, SEPRONA, bomberos, policía local) mediante un flujo de estados trazable. Los ciudadanos consultan el mapa público sin registrarse y, al autenticarse, votan, comentan y siguen los reportes de otros usuarios.

He desarrollado el proyecto en seis *sprints* de una semana con enfoque iterativo e incremental, aplicando *Spec-Driven Development* (SpecKit) para combinar especificaciones formales y asistencia de agentes de IA en el IDE Google Antigravity. El uso de IA está declarado de forma explícita en la memoria.

Tecnológicamente el sistema tiene tres capas en Docker Compose: SPA en React 18 con Tailwind y Leaflet sobre OpenStreetMap (instalable como PWA, con soporte *offline*), API REST en Node.js 20 + Express 4 (con JWT dual token, 2FA TOTP, *rate limiting* y Cloudflare Turnstile), y PostgreSQL 16 con PostGIS 3.4. El modelo tiene 13 entidades.

He definido 46 requisitos funcionales en cinco módulos y 12 requisitos no funcionales. La API expone 35 *endpoints* REST documentados con Swagger. La validación combina Jest+Supertest y Playwright en tres navegadores, sobre un *pipeline* de GitHub Actions.

Los resultados confirman la viabilidad del sistema: despliegue reproducible con un comando, cobertura del *backend* por encima del 60 %, búsquedas geoespaciales en un radio de 5 km por debajo de 500 ms y validación de todos los requisitos funcionales de prioridad alta.

Palabras clave: aplicación cívica, medio ambiente, PostGIS, PWA, React, Node.js, JWT, Docker, Spec-Driven Development.

Abstract

EcoAlerta is a Progressive Web Application (PWA) conceived as a civic social network specialised in environmental issues. It allows any citizen to report environmental incidents (illegal dumping, pollution, wildfires, abandoned or mistreated animals, damage to natural areas, among other categories) with a photo and GPS coordinates. The system classifies each report by category and severity, and delegates it to the appropriate responsible entity (city council, SEPRONA, fire department, local police) through a traceable state flow. Citizens consult the public map without registering and, once authenticated, vote, comment and follow other users' reports.

The project was developed in six one-week sprints following an iterative and incremental approach, applying Spec-Driven Development (SpecKit) to combine formal specifications with AI agent assistance in the Google Antigravity IDE. The use of AI is explicitly declared in the report.

Technologically, the system has three layers deployed with Docker Compose: a SPA in React 18 with Tailwind and Leaflet over OpenStreetMap (installable as a PWA with offline support), a REST API in Node.js 20 + Express 4 (with JWT dual token, TOTP 2FA, rate limiting and Cloudflare Turnstile), and PostgreSQL 16 with PostGIS 3.4. The data model has 13 entities.

46 functional requirements were defined across five modules plus 12 non-functional requirements. The REST API exposes 35 endpoints documented with Swagger. Validation combines Jest+Supertest and Playwright across three browsers, on a GitHub Actions pipeline.

Results confirm the system's viability: reproducible deployment with one command, backend test coverage above 60 %, geospatial searches within a 5 km radius under 500 ms and validation of all high-priority functional requirements.

Keywords: civic application, environment, PostGIS, PWA, React, Node.js, JWT, Docker, Spec-Driven Development.

Índice general

Nota sobre este borrador	iii
Resumen	v
Abstract	vii
1 Introducción	1
1.1 Motivación y contexto	1
1.2 Objetivos	1
1.3 Uso de herramientas de inteligencia artificial	2
1.3.0.0.1 Herramientas.	2
1.3.0.0.2 Tareas asistidas.	2
1.3.0.0.3 Lo que he hecho yo sin delegar.	2
1.3.0.0.4 Auditoría.	3
1.3.0.0.5 Mi posición.	3
1.4 Estructura de la memoria	3
2 Estado del arte	5
2.1 Contexto: aplicaciones cívicas geolocalizadas	5
2.2 Aplicaciones analizadas	5
2.2.1 FixMyStreet	5
2.2.2 Línea Verde	5
2.2.3 iNaturalist	6
2.2.4 SeeClickFix	6
2.2.5 Epicollect5	6
2.3 Comparativa funcional	6
2.4 Tecnologías evaluadas	7
2.5 Posicionamiento de EcoAlerta	8
3 Análisis y diseño	9
3.1 Metodología de desarrollo	9
3.2 Actores del sistema	9
3.3 Requisitos	10
3.3.1 Requisitos funcionales	10
3.3.2 Requisitos no funcionales	11
3.4 Casos de uso	12
3.5 Arquitectura del sistema	14
3.5.1 Stack tecnológico	16
3.6 Modelo de datos	16
3.6.1 Entidades principales	17

3.6.2	Rendimiento geoespacial	18
3.7	Flujo de estados de una incidencia	18
3.8	Diseño de la API REST	19
3.9	Diseño de la interfaz de usuario	20
4	Conclusiones y trabajo futuro	23
4.1	Logros	23
4.1.0.0.1	Transparencia con el uso de IA.	23
4.1.0.0.2	Calidad cercana a producción.	23
4.1.0.0.3	Reusabilidad.	23
4.2	Limitaciones	23
4.2.0.0.1	Cobertura de pruebas.	24
4.2.0.0.2	Sin SMTP real.	24
4.2.0.0.3	Escalado por <i>timeout</i> no programado.	24
4.2.0.0.4	Pruebas de usabilidad limitadas.	24
4.2.0.0.5	Sin prueba de carga.	24
4.2.0.0.6	Moderación social básica.	24
4.3	Trabajo futuro	24
4.4	Reflexión	25

Índice de figuras

3.1	Diagrama de casos de uso. Elaboración propia a partir de RF-01–RF-46. .	13
3.2	Arquitectura de componentes.	15
3.3	Modelo entidad-relación (13 tablas).	17
3.4	Diagrama de estados de la incidencia. Cada transición se guarda en incident_status_history.	19

Índice de tablas

2.1	Comparativa funcional. ✓ sí, ~ parcial, × no.	6
2.2	Decisiones tecnológicas por capa.	8
3.1	Actores del sistema.	10
3.2	Requisitos funcionales (RF-01 a RF-46).	10
3.3	Requisitos no funcionales.	12
3.4	Stack tecnológico.	16
3.5	Endpoints principales.	20

1 Introducción

1.1 Motivación y contexto

España vive una década compleja en lo medioambiental: incendios forestales recurrentes ligados al cambio climático, contaminación atmosférica en ciudades, vertidos en suelos rústicos, maltrato animal. La Ley 7/2022 de residuos, la Ley de bienestar animal y el Pacto Verde Europeo definen un marco exigente, pero su efectividad depende de algo concreto: **detectar a tiempo lo que pasa sobre el terreno**.

Los datos son claros. El Ministerio del Interior registra más de 10 000 incendios forestales al año en España. El SEPRONA tramita más de 130 000 actuaciones anuales por infracciones medioambientales. Una buena parte de estas incidencias llega tarde a los canales oficiales, o no llega.

A la vez, la ciudadanía tiene en el bolsillo lo que necesitaría para reportarlas: el INE estimó en 2023 que el 97,7 % de los hogares españoles tiene Internet y que más del 95 % dispone de un *smartphone* con cámara y GPS. El problema es que los canales tradicionales (teléfono 112, oficinas presenciales, formularios web genéricos sin geolocalización) no aprovechan ese flujo y no devuelven seguimiento al denunciante.

Existen aplicaciones internacionales que sí cubren este hueco. *FixMyStreet* en el Reino Unido lleva desde 2007 acumulando más de dos millones de incidencias reportadas por ciudadanos. *iNaturalist* agrega 180 millones de observaciones de biodiversidad. *Línea Verde* ofrece en España un canal municipal de quejas en más de 300 municipios. El Capítulo 2 las analiza con detalle. Pero ninguna combina, a la vez, reporte geolocalizado con foto, delegación automatizada a la entidad responsable correcta (que no siempre es el ayuntamiento), niveles de severidad ligados al impacto ambiental, y un componente social que ayude a priorizar.

De ahí nace este trabajo. **EcoAlerta** es una aplicación web progresiva (PWA) que combina red social cívica y reporte medioambiental: cualquier ciudadano reporta una incidencia con foto y GPS en menos de un minuto, el sistema la clasifica por categoría y severidad, la delega a la entidad responsable (ayuntamiento, SEPRONA, bomberos, policía local) mediante un flujo de estados trazable, y otros usuarios pueden votar y seguir su evolución. Cualquier persona consulta el mapa público sin registrarse; las acciones con responsabilidad (crear, votar, comentar) requieren cuenta y quedan trazadas.

1.2 Objetivos

El objetivo general del proyecto es:

Diseñar, implementar y validar una aplicación web progresiva que facilite a la ciudadanía el reporte geolocalizado de incidencias medioambientales y permita su delegación automatizada a entidades responsables, con flujo de estados trazable y componente social.

Lo divido en ocho objetivos específicos:

- OBJ-1** Analizar el estado del arte de aplicaciones cívicas medioambientales e identificar huecos de cobertura.
- OBJ-2** Definir una arquitectura escalable basada en contenedores, separando presentación, lógica y datos, con soporte geoespacial.
- OBJ-3** Diseñar un modelo de datos relacional con extensión PostGIS que represente incidencias, fotos, actores, flujos y eventos sociales con *audit trail*.
- OBJ-4** Implementar una API REST documentada con OpenAPI/Swagger, con control de acceso por roles y *rate limiting*.
- OBJ-5** Desarrollar una interfaz web progresiva *mobile-first*, instalable, con soporte *offline* básico, mapa interactivo y conformidad con WCAG 2.1 AA.
- OBJ-6** Incorporar mecanismos de seguridad propios de aplicaciones reales: JWT con rotación de tokens, 2FA, *captcha* invisible, *audit log* y cifrado de secretos.
- OBJ-7** Validar el sistema con una pirámide de pruebas (unitarias, integración, E2E) y pruebas de usabilidad con usuarios reales.
- OBJ-8** Producir documentación reproducible (despliegue con un comando, API documentada, manual de usuario) y un estudio preliminar de modelo de negocio.

1.3 Uso de herramientas de inteligencia artificial

He usado herramientas de IA generativa en este TFG y lo declaro de forma explícita aquí, siguiendo lo que indica la Universidad de Alicante sobre integridad académica cuando la asistencia está permitida y se reconoce. Esta sección recoge qué he usado, para qué, y dónde he tomado yo las decisiones.

1.3.0.0.1 Herramientas. He usado los modelos *Claude* (Anthropic) y *Gemini* (Google), este último integrado en el IDE *Google Antigravity* para programación asistida por agentes.

1.3.0.0.2 Tareas asistidas. La IA me ha ayudado con: generación inicial de *boilerplate* (estructura de rutas Express, componentes React, configuraciones Docker), refactorización de fragmentos ya escritos, generación de pruebas Jest a partir del contrato de los servicios, redacción de borradores de la memoria a partir de la especificación, y revisión ortográfica y de estilo.

1.3.0.0.3 Lo que he hecho yo sin delegar. La idea, el alcance y los objetivos del proyecto. La elección del *stack* y del modelo de datos. El análisis de las cinco aplicaciones de referencia: descarga, uso real y observaciones propias. Las pruebas de usabilidad. La revisión de cada bloque de código generado y su validación con ejecución local. La argumentación de los capítulos de análisis, evaluación y conclusiones.

1.3.0.0.4 Auditoría. El proyecto vive en un repositorio Git público con *commits* en convención *Conventional Commits*, lo que permite reconstruir el proceso. Las especificaciones de cada *sprint* están en `docs/sprints/` y se redactaron antes de pedir asistencia al agente. Asumo la responsabilidad de explicar y justificar en la defensa cualquier decisión de diseño, fragmento de código o párrafo de la memoria, sin distinción de cómo se generó inicialmente.

1.3.0.0.5 Mi posición. El valor del TFG está en tres cosas que la IA no aporta: la identificación del hueco de cobertura que tapa EcoAlerta, el diseño integrado del flujo ciudadano–administrador–entidad, y la ingeniería de requisitos que convierte una idea en 46 RF verificables. La IA es una herramienta de productividad; las decisiones y la responsabilidad son mías.

1.4 Estructura de la memoria

El resto del documento se organiza así. El Capítulo 2 analiza cinco aplicaciones de referencia y compara las tecnologías evaluadas. El Capítulo 3 presenta requisitos, casos de uso, arquitectura, modelo de datos y diseño de la API. El Capítulo ?? describe el desarrollo en seis *sprints*, con extractos de código y decisiones técnicas. El Capítulo ?? recoge resultados de pruebas y validación de requisitos. El Capítulo 4 cierra con logros, limitaciones y trabajo futuro. Los anexos incluyen guía de instalación, manual de usuario y un análisis preliminar de modelo de negocio.

2 Estado del arte

Este capítulo revisa aplicaciones cívicas relevantes para EcoAlerta, compara las tecnologías candidatas para cada capa y justifica las decisiones tomadas en el diseño del Capítulo 3.

2.1 Contexto: aplicaciones cívicas geolocalizadas

Las aplicaciones cívicas con reporte geolocalizado son parte del movimiento *civic tech*. Tres innovaciones las hicieron viables a partir de mediados de los 2000:

1. *Smartphones* con cámara y GPS (iPhone 2007, Android 2008): cualquier ciudadano puede generar evidencia fotográfica geolocalizada en tiempo real.
2. OpenStreetMap (2004): cartografía libre, sin dependencia de proveedores comerciales.
3. Datos abiertos y leyes de transparencia (en España, Ley 19/2013): marco para el intercambio bidireccional ciudadano–administración.

A partir de ahí se consolidan dos familias: las **generalistas** (FixMyStreet, SeeClickFix, que cubren cualquier incidencia urbana) y las **especializadas** (iNaturalist para biodiversidad, Línea Verde para reporte municipal en España). EcoAlerta se sitúa en un hueco intermedio: especializada en medio ambiente, con componente social.

2.2 Aplicaciones analizadas

He seleccionado cinco aplicaciones por relevancia y heterogeneidad: FixMyStreet, Línea Verde, iNaturalist, SeeClickFix y Epicollect5. Las tres primeras las he probado yo mismo en el móvil con la plantilla docs/estado-del-arte/plantilla-analisis.docx. Las dos últimas las analizo desde la documentación.

2.2.1 FixMyStreet

Desarrollada en 2007 por mySociety, una organización británica sin ánimo de lucro de *civic tech*. Código abierto bajo licencia AGPL-3.0. La han desplegado cientos de ayuntamientos, incluida la versión *FixMyStreet Bruxelles* que he usado.

Funcionalidades observadas. **Flujo.** **Tecnología.** Web con diseño *mobile-friendly*, mapas OSM. *Backend* en Perl con Plack y PostgreSQL. **Puntos fuertes.** **Puntos débiles.**

2.2.2 Línea Verde

Servicio comercial de *Línea Verde Configuración y Servicios Medioambientales S.A.*, en más de 300 municipios españoles. Modelo B2G *white-label*: cada ayuntamiento contrata el servicio y obtiene su versión personalizada.

Funcionalidades observadas. Flujo. Tecnología. App móvil nativa (iOS/Android) y portal web. *Backend* no público. **Puntos fuertes. Puntos débiles.**

2.2.3 iNaturalist

Plataforma de ciencia ciudadana para observación de biodiversidad. Nació en 2008 en la Universidad de California; ahora la gestionan California Academy of Sciences y National Geographic Society. Más de 180 millones de observaciones a nivel mundial.

Funcionalidades observadas. Flujo. Tecnología. App móvil nativa y web. Ruby on Rails, PostgreSQL con PostGIS, Elasticsearch. Servicio de reconocimiento de imagen con redes neuronales que sugiere especies. **Puntos fuertes. Puntos débiles.**

2.2.4 SeeClickFix

Fundado en 2008 en New Haven (EE.UU.), referente del reporte cívico americano. Adquirido por CivicPlus en 2019, sigue operando en municipios norteamericanos.

Funcionalidades. Reporte geolocalizado con foto, categorías estandarizadas, flujo de estados (recibido, reconocido, en curso, resuelto), panel administrativo, integración con sistemas *backend* municipales vía API.

Tecnología. App móvil nativa (iOS/Android) y web. API documentada para integraciones.

Relevancia para EcoAlerta. Valida el modelo B2G con flujo de estados trazable. Su limitación es el foco urbano exclusivo: no aborda incendios forestales, fauna silvestre ni daños a espacios naturales.

2.2.5 Epicollect5

Proyecto del Imperial College London para recolección de datos de campo en investigación científica. Usado en epidemiología, conservación y ecología.

Funcionalidades. Formularios personalizados, recolección *offline* con sincronización diferida, exportación en CSV/JSON. Gratuito para investigación.

Tecnología. App móvil y web. *Stack* no documentado públicamente.

Relevancia para EcoAlerta. Su robustez *offline-first* es referencia para el RF-19. Su enfoque genérico lo hace demasiado amplio para el ciudadano medio, pero útil para ONG conservacionistas que pueden configurar formularios específicos.

2.3 Comparativa funcional

Tabla 2.1: Comparativa funcional. ✓ sí, ~ parcial, × no.

Funcionalidad	FixMy-Street	Línea Verde	iNaturalist	SeeClick-Fix	Epi-collect	Eco-Alerta
Reporte geolocalizado con foto	✓	✓	✓	✓	✓	✓

Funcionalidad	FixMy-Street	Línea Verde	iNaturalist	SeeClick-Fix	Epi-collect	Eco-Alerta
Enfoque medioambiental específico	×	~	✓	×	~	✓
Delegación a entidades	~	✓	×	✓	×	✓
Flujo de estados trazable	✓	~	×	✓	×	✓
Componente social (votos, comentarios)	~	×	✓	~	×	✓
Perfil público de usuario	×	×	✓	~	×	✓
Acceso anónimo de consulta	✓	✓	✓	✓	~	✓
Niveles de severidad explícitos	~	×	×	~	✓	✓
<i>Audit trail</i> completo	×	×	×	~	×	✓
PWA instalable	~	×	×	×	~	✓
<i>Offline</i>	×	×	✓	×	✓	✓
2FA / <i>audit</i> de seguridad	×	×	×	×	×	✓
<i>Open source</i>	✓	×	✓	×	~	✓

Ninguna aplicación cubre simultáneamente las seis dimensiones que busca EcoAlerta: enfoque medioambiental, delegación a entidades heterogéneas (no solo municipales), flujo trazable, social significativo, severidad con impacto en priorización y *audit trail*. Ese hueco es el punto de partida del proyecto (OBJ-1).

2.4 Tecnologías evaluadas

Para cada capa consideré varias opciones. La Tabla 2.2 resume las decisiones y por qué.

Tabla 2.2: Decisiones tecnológicas por capa.

Capa	Elección vs alternativas	Razón
Plataforma	PWA vs app nativa	PWA con una sola <i>code base</i> , distribución por URL, instalable. La nativa quedaría como trabajo futuro.
Frontend	React vs Vue, Angular	React tiene la mejor integración con Leaflet (react-leaflet), gran mercado laboral en España, y ya conocía la herramienta.
BD	PostgreSQL+PostGIS vs MongoDB	PostGIS aporta ST_DWithin , índices GIST y transacciones. El modelo relacional encaja mejor con las relaciones del dominio. MongoDB obligaría a denormalizar.
Mapa	Leaflet+OSM vs Google Maps	Leaflet es <i>open source</i> , sin coste ni dependencia comercial. Coherente con la filosofía <i>civic tech</i> de las apps de referencia.
Metodología	SDD vs Scrum clásico	Para un solo desarrollador con asistencia IA, la especificación previa funciona como guía al agente y como base para revisar. Más útil que un Scrum tradicional pensado para equipos.

2.5 Posicionamiento de EcoAlerta

A partir del análisis, EcoAlerta combina elementos de varias fuentes:

- Flujo de estados trazable de FixMyStreet y SeeClickFix, ampliado con validación, marcado de duplicados y escalado por *timeout*.
- Delegación B2G multi-entidad de Línea Verde, generalizada a entidades no municipales (SEPRONA, bomberos, ONG).
- Componente social de iNaturalist (seguimiento, votos, comentarios), adaptado al ámbito medioambiental con la distinción de comentarios oficiales.
- *Offline* robusto al estilo Epicollect5, con Service Worker e IndexedDB.
- Mecanismos de seguridad propios (2FA TOTP, *audit log*, Cloudflare Turnstile) ausentes en las cinco apps analizadas.

Con eso, más la prioridad dada a la accesibilidad WCAG y a la transparencia (código abierto y declaración explícita del uso de IA), se cubre el hueco que las aplicaciones existentes dejan.

3 Análisis y diseño

Este capítulo describe el análisis de requisitos y el diseño técnico de EcoAlerta: metodología, actores, requisitos funcionales y no funcionales, casos de uso, arquitectura, modelo de datos, diseño de la API REST y diseño preliminar de la interfaz. Es la base sobre la que se ha construido la implementación del Capítulo ??.

3.1 Metodología de desarrollo

He organizado el desarrollo en seis *sprints* de una semana, con enfoque iterativo e incremental. La razón es práctica: los requisitos cambian con cada tutoría, conviene tener un producto funcional pronto para recibir realimentación, y el calendario disponible es ajustado.

Sobre los *sprints* aplico **Spec-Driven Development** (SpecKit) (**speckit**), con cuatro fases por iteración:

1. **Specify**: redacto la especificación funcional del *sprint* y los criterios de aceptación.
2. **Plan**: descompongo en tareas concretas con dependencias.
3. **Implement**: ejecuto las tareas con asistencia del agente de IA en Google Antigravity.
4. **Verify**: pruebas unitarias, de integración y E2E, y validación de los criterios.

SDD me sirve por una razón específica: como uso asistencia de IA (sección 1.3), necesito una especificación previa que sirva tanto de guía al agente como de referencia para revisar lo que produce. El documento maestro `EcoAlerta_Development_Spec_v1.md` vive en el repositorio como fuente de verdad. He tenido tutorías con el tutor cada dos semanas; las actas están en `docs/actas-tutorias/`.

3.2 Actores del sistema

Cinco actores con sus accesos:

Tabla 3.1: Actores del sistema.

Actor	Descripción	Autenticación
Ciudadano anónimo	Consulta el mapa y el detalle de incidencias. No puede crear, votar ni comentar.	Ninguna
Ciudadano registrado	Crea incidencias, vota, comenta y sigue reportes. Tiene perfil social.	JWT
Administrador	Valida, asigna a entidades y cambia estados. Accede al <i>dashboard</i> .	JWT (rol <code>admin</code>)
Entidad responsable	Ayuntamiento, SEPRONA, bomberos, policía local u ONG. Recibe asignaciones.	JWT (rol <code>entity</code>)
Sistema	Notificaciones automáticas, escalado por <i>timeout</i> y <i>audit log</i> .	Interno

La separación entre anónimo y registrado es deliberada: el mapa público funciona como herramienta de transparencia cívica abierta a cualquiera, y al mismo tiempo las acciones con responsabilidad (crear, votar, comentar) requieren cuenta para que queden trazadas.

3.3 Requisitos

He definido 46 requisitos funcionales, agrupados en cinco módulos, y 12 requisitos no funcionales. La prioridad funcional es *Alta* (sistema mínimo viable), *Media* (importante para experiencia) o *Baja* (deseable).

3.3.1 Requisitos funcionales

Tabla 3.2: Requisitos funcionales (RF-01 a RF-46).

ID	Pri.	Descripción	Actor
Módulo de autenticación			
RF-01	Alta	Registro con email y contraseña.	Ciudadano
RF-02	Alta	Login con tokens JWT <i>access</i> y <i>refresh</i> .	Todos
RF-03	Alta	Acceso anónimo en lectura (mapa y listado).	Anónimo
RF-04	Alta	Roles: <code>citizen</code> , <code>admin</code> , <code>entity</code> , <code>moderator</code> .	Sistema
RF-05	Baja	Recuperación de contraseña por email (limitada por SMTP).	Ciudadano
RF-06	Media	Edición de perfil.	Ciudadano
RF-07	Media	Perfil público con incidencias del usuario.	Ciudadano
Módulo de incidencias			
RF-10	Alta	Creación con título, descripción, categoría, fotos y GPS.	Ciudadano reg.
RF-11	Alta	Captura automática de la ubicación GPS.	Sistema
RF-12	Alta	Selección manual en el mapa si no hay GPS.	Ciudadano
RF-13	Alta	Clasificación por categoría (12 predefinidas).	Sistema
RF-14	Alta	Severidad: leve, moderado, grave, crítico.	Ciud./Admin

ID	Pri.	Descripción	Actor
RF-15	Alta	1–5 fotos por incidencia, máx. 10 MB cada una.	Ciudadano
RF-16	Media	<i>Thumbnails</i> automáticos.	Sistema
RF-17	Media	Edición/eliminación solo si <code>status = pending</code> .	Ciudadano
RF-18	Media	Historial de cambios en <code>incident_status_history</code> .	Sistema
RF-19	Baja	Borradores con capacidad <i>offline</i> (PWA).	Ciudadano
Módulo de mapa interactivo			
RF-20	Alta	Mapa con todas las incidencias geolocalizadas.	Todos
RF-21	Alta	Agrupación en <i>clusters</i> al disminuir el zoom.	Sistema
RF-22	Alta	Filtros por categoría, severidad, estado, fecha y distancia.	Todos
RF-23	Alta	Marcadores coloreados por severidad.	Sistema
RF-24	Media	Búsqueda por dirección (<i>geocoding</i>).	Todos
RF-25	Alta	Detalle al pulsar el marcador.	Todos
RF-26	Media	Búsquedas por radio (<code>ST_DWithin</code>).	Sistema
Módulo de administración y delegación			
RF-30	Alta	Panel admin con <i>dashboard</i> de estadísticas.	Admin
RF-31	Alta	Cambio de estado: <code>pending</code> → <code>validated</code> → <code>in_progress</code> → <code>resolved/rejected</code> .	Admin
RF-32	Alta	Asignación a entidades responsables.	Admin
RF-33	Alta	Notificación a la entidad y actualización de estado.	Entidad
RF-34	Media	Escalado automático por <i>timeout</i> configurable.	Sistema
RF-35	Media	Estadísticas por categoría, estado, zona, tiempo de resolución.	Admin
RF-36	Media	Gestión de categorías y entidades.	Admin
RF-37	Baja	Marcado de duplicadas con vinculación.	Admin
Módulo social y notificaciones			
RF-40	Alta	Votación de incidencias.	Ciudadano
RF-41	Alta	Comentarios con distintivo de oficiales.	Ciud./Admin/Ent.
RF-42	Alta	Notificación al autor y seguidores en cambio de estado.	Sistema
RF-43	Media	Seguimiento de incidencias.	Ciudadano
RF-44	Media	Perfil de usuario con incidencias, votos y estadísticas.	Ciudadano
RF-45	Media	Notificaciones <i>push</i> con Service Worker.	Sistema
RF-46	Baja	Notificaciones por email para cambios importantes.	Sistema

3.3.2 Requisitos no funcionales

Los RNF son atributos de calidad medibles. Cada uno se valida con métricas en el Capítulo ??.

Tabla 3.3: Requisitos no funcionales.

ID	Categoría	Descripción
RNF-01	Rendimiento	Tiempo de respuesta API < 500 ms para el 95 % de peticiones.
RNF-02	Rendimiento	Soporte de al menos 100 usuarios simultáneos.
RNF-03	Usabilidad	<i>Responsive</i> : 320 px, 768 px, ≥ 1024 px.
RNF-04	Portabilidad	PWA instalable con <code>manifest.json</code> y Service Worker.
RNF-05	Seguridad	Contraseñas con <i>bcrypt</i> (mínimo 10 rondas).
RNF-06	Seguridad	<i>Rate limiting</i> en todos los <i>endpoints</i> .
RNF-07	Seguridad	HTTPS obligatorio en producción.
RNF-08	Calidad	Cobertura de pruebas unitarias > 70 % en <i>backend</i> .
RNF-09	Calidad	Tests E2E para los 3 flujos principales.
RNF-10	Mantenibilidad	Despliegue reproducible con un comando (<code>docker compose up</code>).
RNF-11	Mantenibilidad	Documentación Swagger/OpenAPI.
RNF-12	Accesibilidad	WCAG 2.1 nivel AA.

3.4 Casos de uso

La Figura 3.1 muestra los casos de uso por módulo. <<requiere>> marca los que necesitan autenticación; <<include>> marca los que disparan otro caso (por ejemplo, validar siempre incluye notificar al autor).

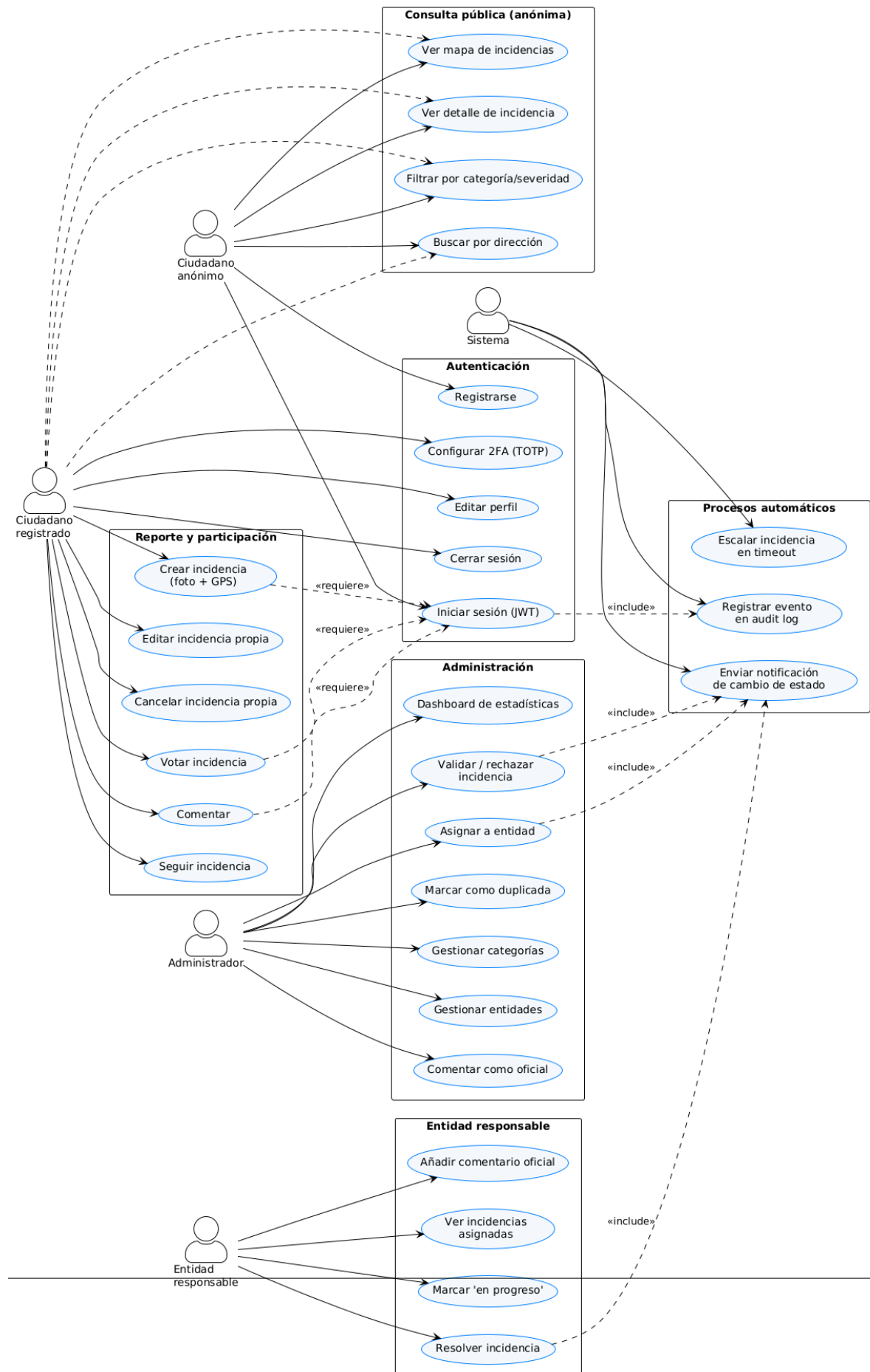


Figura 3.1: Diagrama de casos de uso. Elaboración propia a partir de RF-01–RF-46.

El actor *Sistema* agrupa los procesos sin intervención humana directa: notificaciones tras cambios de estado (RF-42), escalado por *timeout* (RF-34) y *audit log* de seguridad.

3.5 Arquitectura del sistema

Arquitectura cliente-servidor de tres capas:

1. **Presentación:** SPA React 18 con Tailwind y Leaflet. Instalable como PWA.
2. **Lógica:** API REST en Node.js 20 + Express 4, con controladores, servicios, *middlewares* y validadores. Swagger en `/api/docs`.
3. **Datos:** PostgreSQL 16 con PostGIS 3.4 (tipos geométricos, `ST_DWithin`, índices GIST).

Sobre las tres capas, un *reverse proxy* nginx termina HTTPS, sirve los estáticos del *frontend* y redirige `/api/*` al *backend*. Todo se despliega con Docker Compose, que da reproducibilidad (RNF-10) y simplifica la demo de la defensa.

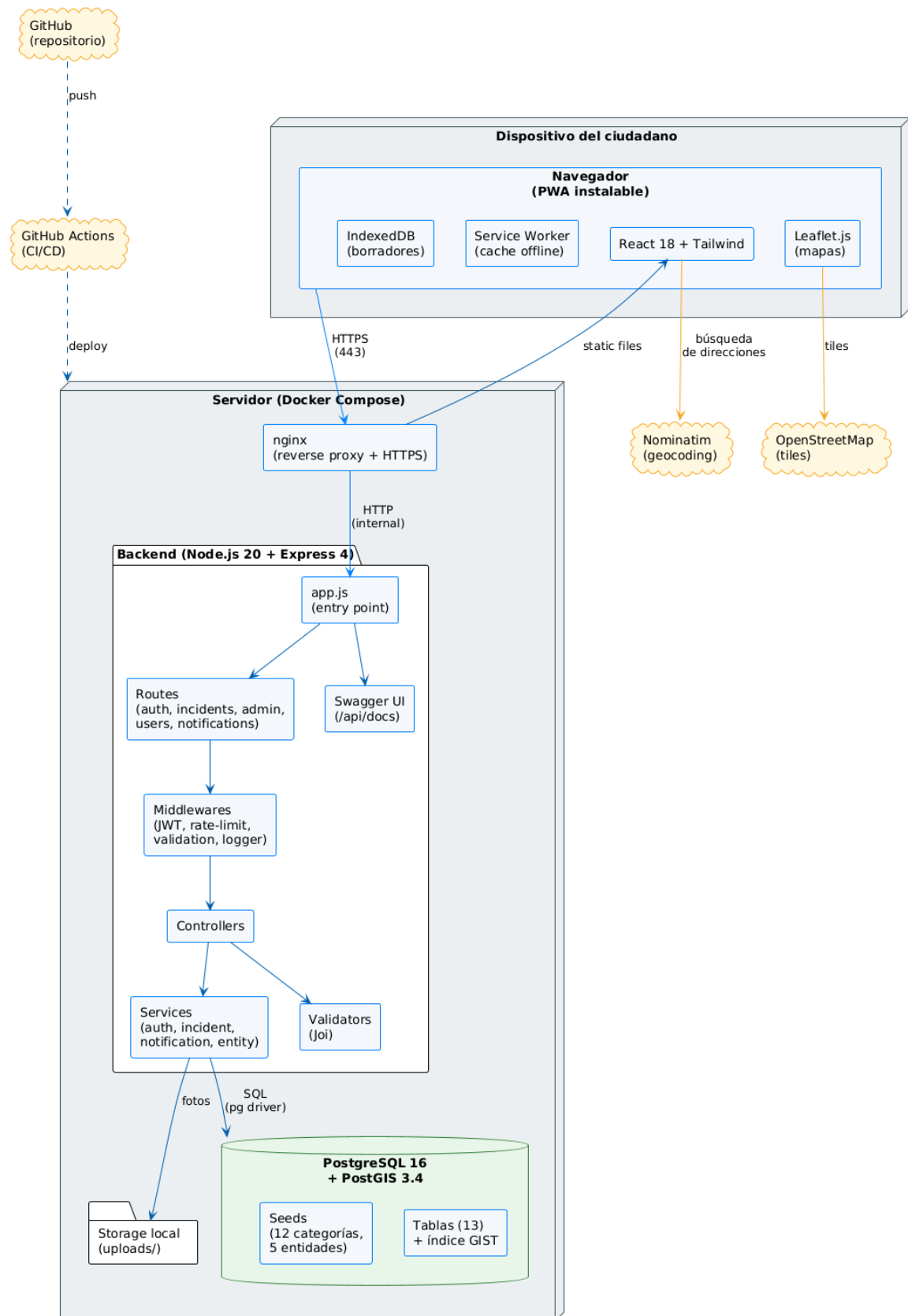


Figura 3.2: Arquitectura de componentes.

Servicios externos: *tiles* de OpenStreetMap y *geocoding* contra Nominatim. Ambos gratuitos; evitan dependencias de proveedores comerciales con API de pago, lo que simplifica el modelo de negocio del Anexo ??.

3.5.1 Stack tecnológico

El tutor me dio libertad para elegir el *stack*, con la condición de razonarlo. La Tabla 3.4 resume las tecnologías y la justificación breve de cada elección.

Tabla 3.4: Stack tecnológico.

Capa	Tecnología	Justificación
<i>Frontend</i>	React 18 + Tailwind	Maduro, integración con Leaflet, <i>responsive</i> sencillo.
Mapas	Leaflet + OSM	<i>Open source</i> , sin API key ni cuota.
PWA	Service Worker	Instalable, caché <i>offline</i> (RF-19, RNF-04).
<i>Backend</i>	Node.js 20 LTS + Express 4	Asíncrono, ecosistema amplio, LTS hasta 2026.
BD	PostgreSQL 16 + PostGIS 3.4	Tipos geométricos, <i>ST_DWithin</i> , índices GIST (RF-26).
Auth	JWT (access+refresh) + bcrypt	Stateless, escalable, RNF-05.
Contenedores	Docker Compose	Despliegue con un comando (RNF-10).
CI/CD	GitHub Actions	Gratuito, <i>lint</i> y tests por <i>push</i> .
Pruebas	Jest + Supertest + Playwright	Pirámide unit/integration/E2E (RNF-08, RNF-09).
<i>Logging</i>	Winston	Estructurado JSON.
API docs	Swagger	Anotaciones en código, sin desincronización (RNF-11).

3.6 Modelo de datos

El modelo tiene 13 entidades, agrupadas en cuatro subsistemas:

- **Identidad y seguridad:** `users`, `user_2fa`, `user_recovery_codes`, `security_audit_log`.
- **Catálogos:** `categories`, `responsible_entities`.
- **Núcleo de incidencias:** `incidents`, `incident_photos`, `incident_comments`, `incident_status_hist`.
- **Interacción social y notificaciones:** `incident_votes`, `incident_follows`, `notifications`.

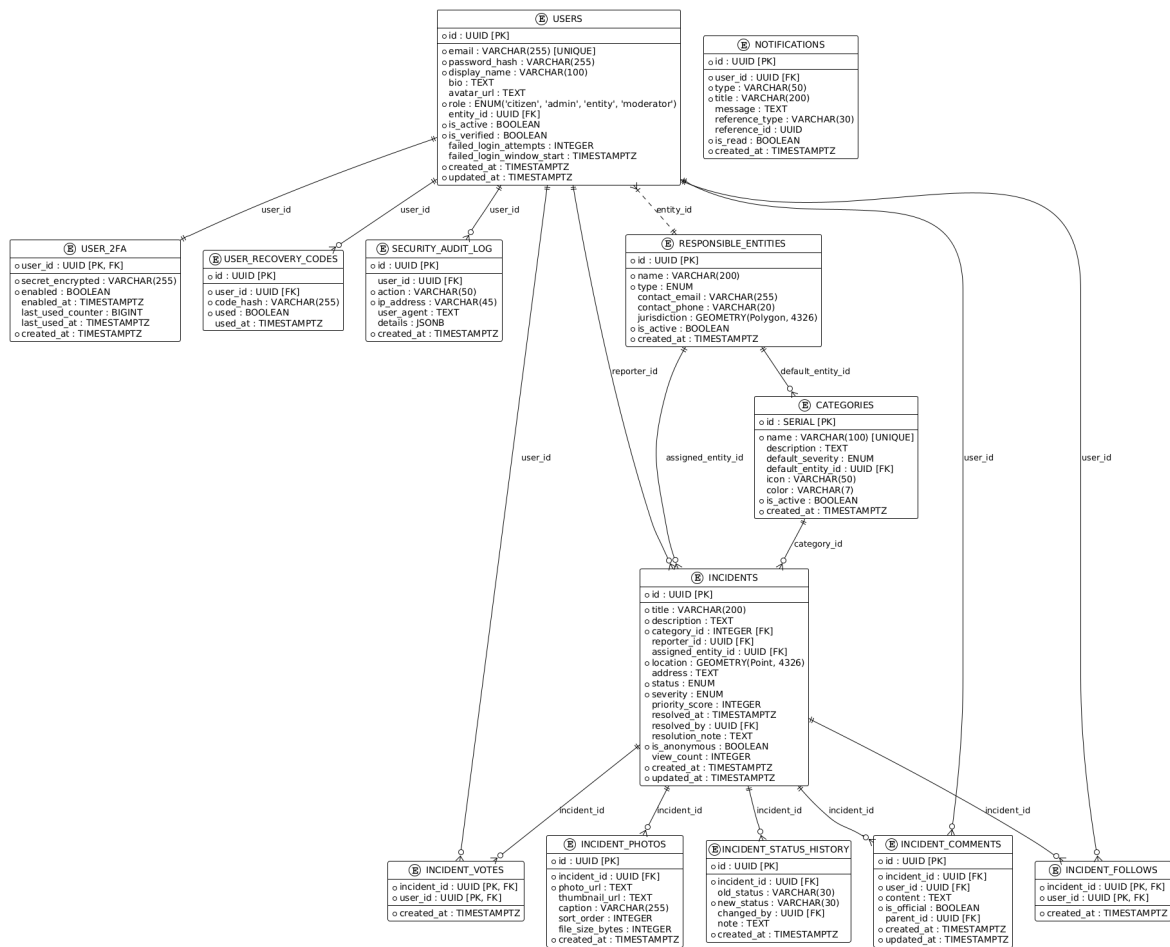


Figura 3.3: Modelo entidad-relación (13 tablas).

3.6.1 Entidades principales

users. Información de los usuarios, con `role`, `entity_id`, `is_active` e `is_verified`. Los contadores `failed_login_attempts` y `failed_login_window_start` dan *rate limit* por usuario que refuerza RNF-06.

user_2fa, **user_recovery_codes.** Añadidas en el Sprint 6. Habilitan 2FA TOTP y guardan códigos de recuperación cifrados.

security_audit_log. Eventos de seguridad (login OK/KO, cambios de rol y contraseña) con IP, `user_agent` y JSONB. No se borra: es el *audit trail* del sistema.

responsible_entities. Catálogo de organismos asignables. `jurisdiction` es `GEOMETRY(Polygon, 4326)` para asignación geoespacial automática a futuro.

categories. Las 12 categorías predefinidas, con severidad por defecto y entidad responsable sugerida. Reduce fricción al reportar.

incidents. Entidad central. `location` es `GEOMETRY(Point, 4326)` con índice GIST para consultas geoespaciales (RF-26). `priority_score` se deriva de severidad y votos para ordenar el *dashboard* (RF-35).

incident_photos, incident_comments, incident_votes, incident_follows. Relaciones 1:N (fotos, comentarios) y N:M (votos, seguimientos). Las dos últimas usan PK compuesta (**incident_id, user_id**) para garantizar unicidad sin índice extra.

incident_status_history. Historial inmutable de transiciones (RF-18). Cada cambio inserta una fila con estado anterior, nuevo, usuario y nota opcional.

notifications. Por usuario. **reference_type** y **reference_id** actúan como *soft foreign key* polimórfica.

3.6.2 Rendimiento geoespacial

Las consultas más frecuentes son del tipo «incidencias en un radio de X km». Las resuelvo con **ST_DWithin** sobre un índice GIST:

 SQL

```
1 CREATE INDEX idx_incidents_location ON incidents USING GIST (location);
2
3 SELECT id, title, severity FROM incidents
4 WHERE ST_DWithin(location::geography,
5                  ST_MakePoint(:lng, :lat)::geography,
6                  :radius_meters);
```

La proyección a **geography** antes de **ST_DWithin** hace que la distancia se evalúe en metros sobre la superficie terrestre, no en grados sobre el plano cartesiano (que daría resultados incorrectos fuera del ecuador).

3.7 Flujo de estados de una incidencia

Cada incidencia pasa por un ciclo de vida acotado, implementado como **ENUM** en PostgreSQL.

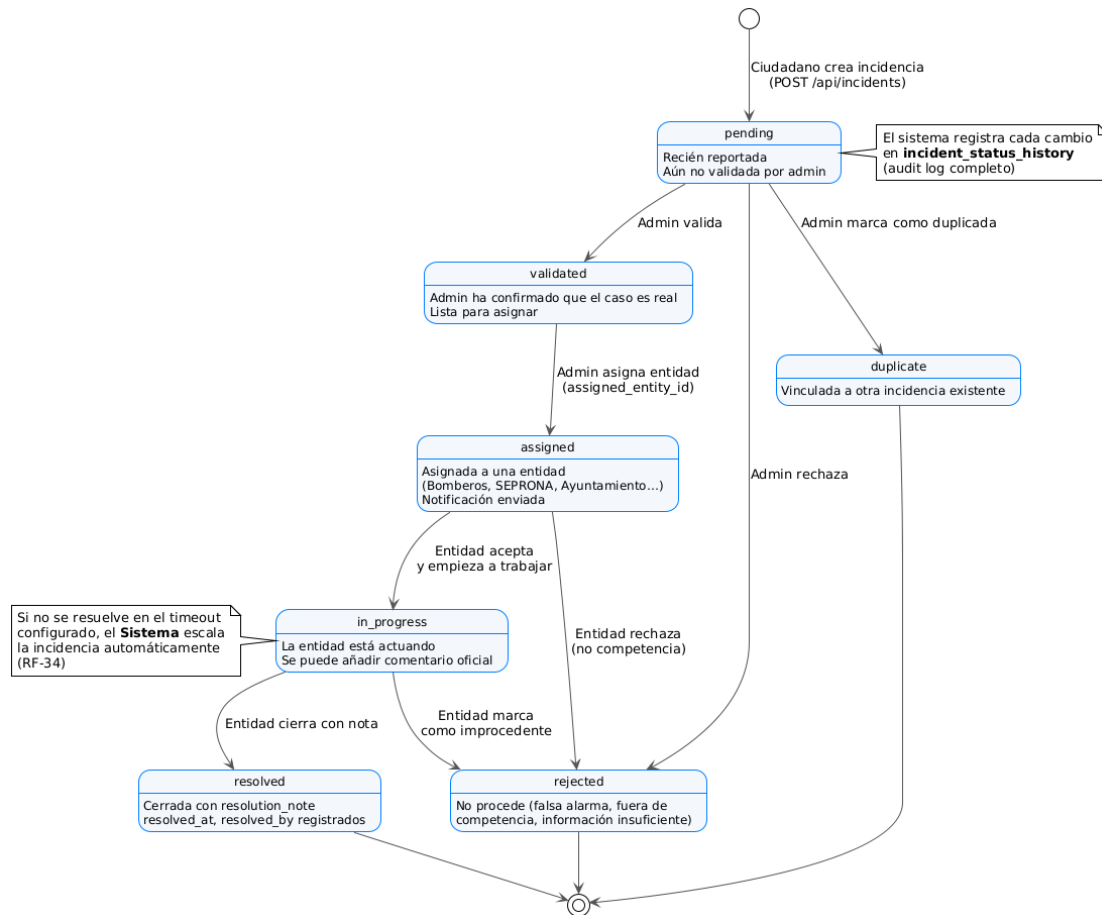


Figura 3.4: Diagrama de estados de la incidencia. Cada transición se guarda en `incident_status_history`.

El estado inicial es **pending**. Desde ahí, el admin valida, rechaza o marca como duplicada. Tras validar, asigna a una entidad y la incidencia pasa a **assigned** (con notificación automática al autor, RF-42). La entidad confirma el inicio del trabajo (**in_progress**) y al actuar la cierra como **resolved** con nota obligatoria. Como excepción, una entidad puede rechazar si no es de su competencia. El RF-34 contempla escalado automático por *timeout* si una incidencia en **in_progress** no avanza.

3.8 Diseño de la API REST

La API expone 35 *endpoints* bajo `/api/v1`, organizados en seis recursos. Documentación interactiva Swagger en `/api/docs`.

Tabla 3.5: Endpoints principales.

Método	Ruta	Descripción
<i>Auth</i>		
POST	/auth/register	Registro.
POST	/auth/login	Login (<i>access+refresh</i>).
POST	/auth/refresh	Renovar <i>access token</i> .
GET/PUT	/auth/me	Perfil del usuario autenticado.
<i>Incidents</i>		
GET	/incidents	Listado paginado con filtros.
GET	/incidents/:id	Detalle con fotos y comentarios.
POST	/incidents	Creación (<i>multipart</i>).
PUT/DELETE	/incidents/:id	Edición/baja (autor o admin).
GET	/incidents/nearby	Geoespacial (<i>lat, lng, radius</i>).
GET	/incidents/:id/history	Historial de estados.
POST	/incidents/:id/vote	Votar.
POST	/incidents/:id/follow	Seguir.
POST	/incidents/:id/comments	Comentar.
<i>Admin</i>		
GET	/admin/dashboard	Estadísticas.
GET/PUT	/admin/incidents	Listado y cambios de estado/asignación.
GET/PUT	/admin/users	Gestión de usuarios y roles.
GET/POST/PUT	/admin/entities	CRUD de entidades responsables.
<i>Users / Notifications / Health</i>		
GET	/users/:id/*	Perfil público, incidencias, estadísticas.
GET/PUT	/notifications	Listar y marcar como leídas.
GET	/health	<i>Health check</i> .

La autenticación usa el esquema **Bearer** sobre **JWT**. *Access token* de 15 minutos, *refresh token* de 7 días con rotación en cada uso. El *middleware* verifica firma y vigencia antes de poblar `req.user`.

3.9 Diseño de la interfaz de usuario

Tres principios:

1. **Mobile-first**: el caso de uso principal (reportar sobre el terreno) ocurre desde el móvil. Los *breakpoints* de Tailwind adaptan la UI a pantallas mayores (RNF-03).

2. **Mapa como eje:** la pantalla principal es el mapa, no una lista. Transmite densidad y distribución del problema, coherente con la naturaleza cívica del producto.
3. **Accesibilidad:** contrastes verificados, navegación por teclado y etiquetas ARIA en los controles (RNF-12).

Las pantallas clave son: mapa principal, formulario de reporte, detalle de incidencia, panel admin y perfil de usuario. Las capturas de la aplicación real están en el Capítulo ?? (sección ??) y en el manual del Anexo ??.

4 Conclusiones y trabajo futuro

4.1 Logros

Repasando los ocho objetivos planteados en la sección 1.2:

- OBJ-1.** Cumplido. Capítulo 2 con análisis de cinco aplicaciones y tabla comparativa.
- OBJ-2.** Cumplido. Arquitectura en tres capas operativa, despliegue con un solo comando.
- OBJ-3.** Cumplido. Modelo de datos con 13 entidades e índice GIST activo.
- OBJ-4.** Cumplido. 35 *endpoints* REST documentados con Swagger y control por rol.
- OBJ-5.** Cumplido. Interfaz *mobile-first* instalable como PWA, *breakpoints* verificados.
- OBJ-6.** Cumplido. 2FA, Turnstile, *audit log* y cifrado de secretos integrados (Sprint 6).
- OBJ-7.** Parcialmente cumplido. Pirámide de pruebas operativa; las pruebas de usabilidad son preliminares.
- OBJ-8.** Cumplido. Documentación reproducible y modelo de negocio en el Anexo ??.

Tres puntos del proyecto me parecen especialmente destacables:

4.1.0.0.1 Transparencia con el uso de IA. El proyecto es coherente con el contexto en el que se desarrolla. Existen herramientas de asistencia y las he usado, declarándolo y dejando trazas auditables. La sección 1.3 y el Capítulo ?? entran en detalle. Creo que esa actitud es la única razonable a día de hoy.

4.1.0.0.2 Calidad cercana a producción. El sistema incluye CI/CD con GitHub Actions, *audit log* de seguridad, documentación API autogenerada, capacidad *offline* y 2FA. Nada de esto era estrictamente necesario para cumplir el enunciado, pero da al resultado un acabado más profesional.

4.1.0.0.3 Reusabilidad. El modelo de datos, el flujo de estados y el patrón de delegación a entidades pueden trasladarse con pocos cambios a dominios cercanos: seguridad ciudadana, patrimonio, salud pública. El diseño no se queda en lo medioambiental.

4.2 Limitaciones

Reconocer lo que no está bien terminado es parte del trabajo:

4.2.0.0.1 Cobertura de pruebas. La cobertura de *branches* (38,91 %) está claramente por debajo del 70 % que fijé en RNF-08. Muchas ramas de error no se prueban. Es un trabajo mecánico que no me dio tiempo a cerrar.

4.2.0.0.2 Sin SMTP real. La recuperación de contraseña por email (RF-05) y las notificaciones por email (RF-46) están programadas pero no se han probado contra un servidor real. Funcionarán al configurar las variables de entorno en producción.

4.2.0.0.3 Escalado por *timeout* no programado. El RF-34 (escalado de incidencias no atendidas) está diseñado pero no hay un *scheduler* corriendo. Falta añadir cron o un *worker* dedicado.

4.2.0.0.4 Pruebas de usabilidad limitadas. Tres usuarios bastan para detectar problemas graves de usabilidad, pero no son una muestra robusta. Una evaluación más sólida necesitaría 8–15 personas.

4.2.0.0.5 Sin prueba de carga. El RNF-02 (100 usuarios simultáneos) lo he validado por dimensionado teórico, no con *k6* o *Artillery* contra el sistema real.

4.2.0.0.6 Moderación social básica. Los comentarios entre usuarios funcionan, pero no hay filtros de lenguaje, reputación ni *shadow banning*. En un despliegue público abierto haría falta.

4.3 Trabajo futuro

Las líneas que veo claras como continuación natural:

Corto plazo. Subir la cobertura por encima del 80 %, desplegar en *cloud* con HTTPS real (Let's Encrypt) y backups automatizados, programar el *scheduler* de RF-34, y pasar pruebas de carga con *k6* para validar RNF-02.

Medio plazo. Una aplicación nativa complementaria con React Native, aprovechando la API REST existente, para mejorar las notificaciones *push* y el acceso a sensores. Asignación automática de incidencias a entidades por intersección geoespacial entre ubicación y jurisdicción (el campo *jurisdiction* ya existe en el modelo). Sistema de reputación del ciudadano basado en histórico. Clasificación automática de la fotografía mediante visión por computador, que sugiera categoría y severidad al crear la incidencia.

Largo plazo. Convenios con ayuntamientos piloto en la provincia de Alicante para validar el producto en producción. Integración con plataformas de gestión municipal (GESCON, Gestiona). Modelo de negocio B2G *freemium* desarrollado a partir del análisis del Anexo ???. Apertura del código con licencia MIT o Apache 2.0 para fomentar replicación en otros municipios.

4.4 Reflexión

El TFG me ha permitido juntar en un único producto cosas que vimos en distintas asignaturas del Grado: bases de datos, programación web, arquitectura, ingeniería de requisitos, pruebas, seguridad, gestión de proyectos. Hasta ahora cada una vivía en su silo; aquí se han tenido que coordinar.

En lo metodológico, combinar SDD con agentes de IA en un IDE como Antigravity ha sido un aprendizaje en sí mismo. Imagino que es como va a trabajar buena parte de la industria de aquí a unos años. La conclusión que me llevo es que la diferencia no la marca delegar, sino dirigir bien: especificar con precisión, revisar con rigor y firmar el resultado.

Quiero agradecer al tutor José Luis Sánchez Romero su disponibilidad y la libertad que me dio para elegir el *stack* y el alcance. Sin esa libertad, EcoAlerta no habría salido como ha salido.
